Deep Generative Models

VAE and GAN models

VAE and GAN models are the **foundations of generative AI**

Generative and discriminative models

We should distinguish between **generative** and **discriminative models**

Generative model

Statistical models of the **data distribution** p_X or p_{XY}

- Depends on the availability of target data

Discriminative model

Statistical models of the **conditional distribution** $p_{Y|X}$

- Can be built **from a generative model via the Bayes rule**

$$p_{Y|X}(y \mid x) = \frac{p_{XY}(x, y)}{\sum_{y'} p_{XY}(x, y')}$$

Density estimation

We should distinguish between **explicit** and **implicit density estimation**

Explicit density estimation

We want to **define and compute the probability distribution that fits the data**

Finds a probability distribution $f \in \Delta(Z)$ such that:

- f describes the distribution of data $z \in Z$
- Data z are originally described by the unknown probability distribution $p_{data} \in \Delta(Z)$

Implicit density estimation

We **want to create examples that looks to be computed by a given probability distribution**

Finds a function $f \in Z^{\Omega}$ such that:

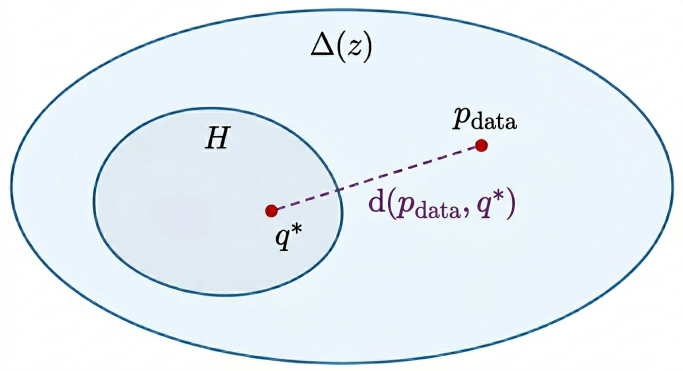
- Takes as input an object ω and generates data $f(\omega) \in Z$
- Data $f(\omega)$ implicitly adapts to the unknown probability distribution $p_{data} \in \Delta(Z)$

Objective

The **objective** is to:

- **Define** the **hypothesis space** $H \subset \Delta(Z)$
  - Set of possible models to represent the probability distributions
- **Choose** a **divergence measure** d
  - To measure the error between 2 probability distributions
- **Minimize** the **error**
  - To find the **ideal model** q^* that fits p_{data} the best

$$q^* \in \arg\min_{q \in H} d(p_{data}, q)$$

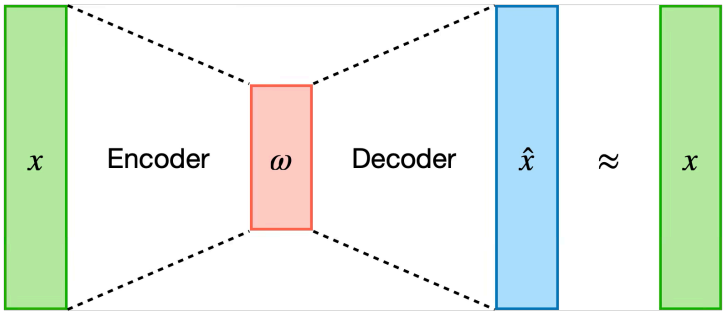


VAE - Variational Autoencoders

Autoencoders

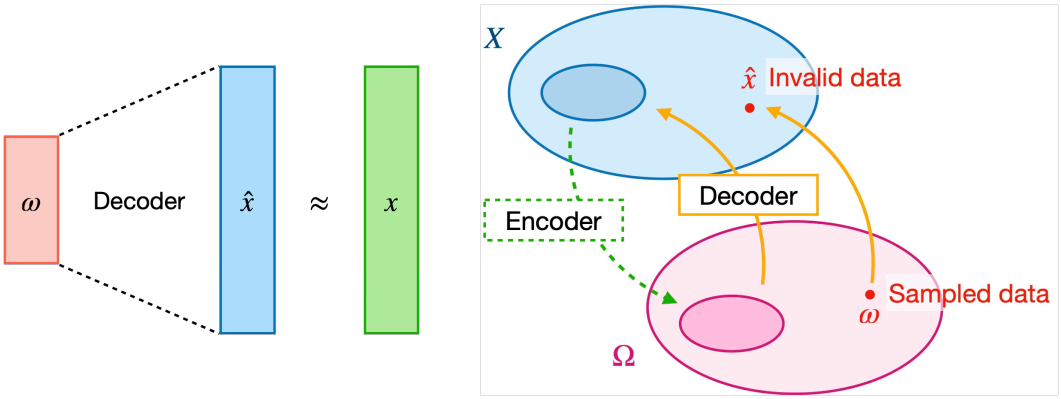
An **autoencoder** is a technique composed of:

- **Encoder**: compresses an input $x \in X$ to an entity $\omega \in \Omega$ (Ω is said latent space)
- **Decoder**: rebuilds from ω the original data, obtaining $\hat{x}$



We can use the decoder component to **generate new data** $\hat{x}$

- From a given compressed input $\omega \in \Omega$
- However **it won't generate data according to the data distribution** p_{data}
  - Why? The **latent space** Ω **isn't properly structured**



⚠️ The decoder **isn't a generative model**

- We want to train a generative model based on **explicit density estimation**

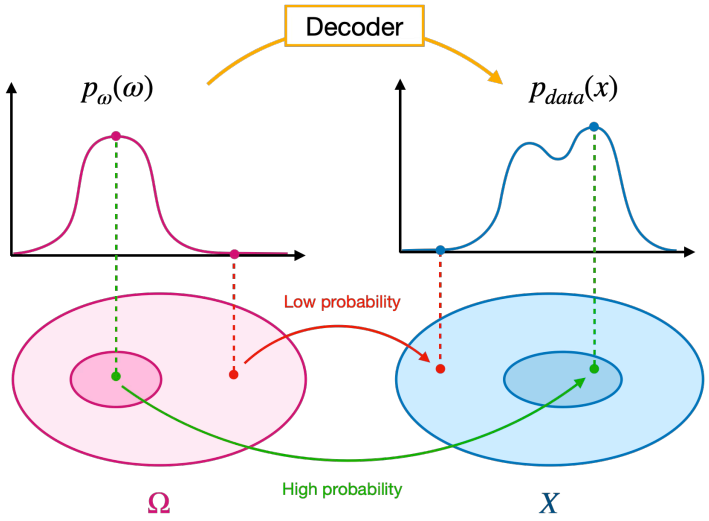
VAE intuition

The **VAE encoder maps** the **input** x **to** a chosen **probability distribution** p_ω (= prior distribution)

- **Usually** a **Gaussian distribution**, returns μ mean and covariance Σ

💡 We **want** the **decoder** to **manipulate** the **prior distribution** p_ω **and approximate** the p_{data}

- If we sample a **highly probable point from** p_ω → map it to a **highly probable point in** p_{data}
- If we sample a **lowly probable point from** p_ω → map it to a **lowly probable point in** p_{data}



In formulas we **aim** to **compute** $q_\theta(x)$

- It's the **global probability that our model generates the output** x → aim too **maximize it**

$$q_\theta(x) = \mathbb{E}_{\omega \sim p_\omega}[q_\theta(x \mid \omega)]$$

where $q_\theta(x \mid \omega)$ is the **decoder** (= probability to generate x given ω)

- We want it **to be as similar as** p_{data} **as possible**
- We need to **find the optimal parameters** θ
  such that the **divergence between** q_θ **and** p_{data} **is minimized**

$$\theta^\star \in \arg\min d(p_{data}, q_\theta)$$

We can define d to be the **Kullback-Leibler** divergence d_{KL}

Intractable objective

We have seen that $q_\theta(x) = \mathbb{E}_{\omega \sim p_\omega}[q_\theta(x \mid \omega)]$

However, it is the **sum of all the possible probabilities**
computed by the decoder **on all possible** ω

- Is an **infinite calculation** → it **can't be computed**

Variational Bound

However we observe that:

$$\begin{aligned}
\log \mathbb{E}_{\omega \sim p_\omega}[q_\theta(x \mid \omega)] &= \log \mathbb{E}_{\omega \sim q_\psi(\cdot \mid x)} \left[q_\theta(x \mid \omega) \frac{p_\omega(\omega)}{q_\psi(\omega \mid x)} \right] \\
&\geq \mathbb{E}_{\omega \sim q_\psi(\cdot \mid x)} \left[\log \left(q_\theta(x \mid \omega) \frac{p_\omega(\omega)}{q_\psi(\omega \mid x)} \right) \right] \\
&= \underbrace{\mathbb{E}_{\omega \sim q_\psi(\cdot \mid x)}[\log q_\theta(x \mid \omega)]}_{\text{Reconstruction}} - \underbrace{d_{KL}(q_\psi(\cdot \mid x), p_\omega)}_{\text{Regularizer}}
\end{aligned}$$

Where $q_\psi(\omega \mid x) \in \Delta(\Omega)$ is the **probability distribution generated by the encoder**

As a cons, **to maximize the distribution** we can **maximize it's lower bound**

Variational Bound Loss

We **can't minimize** d_{KL} directly, but **we can minimize it's upper bound**:

$$d_{KL}(p_{data}, q_\theta) \leq \mathbb{E}_{x\sim p_{data}}\Big[\underbrace{-\mathbb{E}_{\omega\sim q_\psi(\cdot|x)}[\log q_\theta(x\,|\,\omega)]}_{\text{Reconstruction}} + \underbrace{d_{KL}(q_\psi(\,\cdot\,|\,x), p_\omega)}_{\text{Regularizer}}\Big] + \text{const}$$

The **reconstruction**:

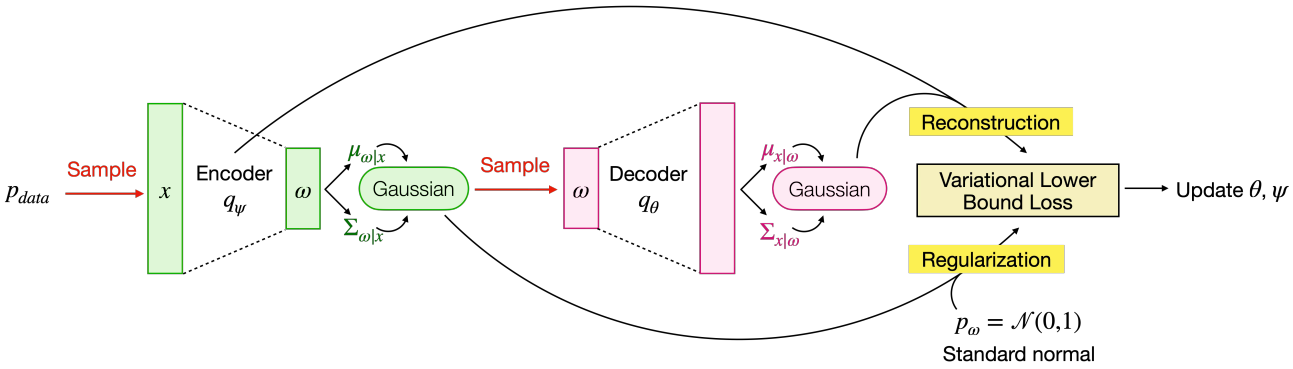
- Measures **how good the decoder rebuilds** x given ω
- Is **still intractable**, **but** encoder and decoder **parameters can be computed**

The **regularizer**:

- **Makes sure** the **distribution generated** by the **encoder** has an a-priori (**Gaussian**) **set distribution**
- Can have a closed-form solution

Training phase

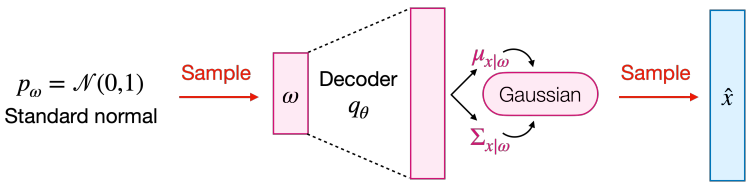
Based on the previous observations, we can define the training process of VAE



Schema of the training process

Inference phase

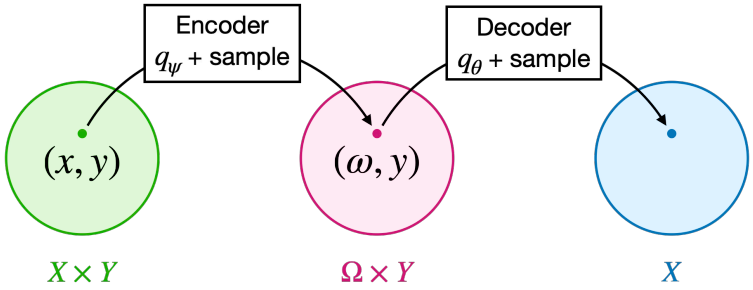
Based on the previous observations, we can define the inference process of VAE



Schema of the inference process

Conditional VAE

The **new data** can even be **generated conditioned by** some **side information** $y \in Y$



The model should define:

- **Encoder distribution** as $q_\psi(\omega \mid x, y)$
- **Decoder distribution** as $q_\theta(\omega \mid x, y)$
- **Prior distribution** as $p_\omega(\omega \mid y)$

Advantages and issues

VAEs have both advantages and issues

Advantages

- **Well** structured and **behaved latent space**
  - Forced to **follow** a **known distribution** → **doesn't** have **empty areas**
  - **Allows** to **smoothly moves in** the **latent space**→ to create interpolations between images
- **Fast inference** time
  - The generation of new data **only requires to pass samples through the decoder**

Issues

- **Underfitting** in **early training phase**
  - The **regularization** term might be **too strong**
    → the **model struggles** to **rebuild** data
- Tends to **generate blurry data**
  - The model **aims** to **minimize** the **mean value error**
    → produces data close to the mean value

GAN - Generative Adversarial Networks

GAN intuition

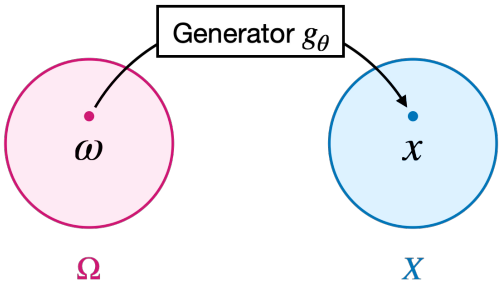
GANs are based on **implicit density estimation**

- The aim is to **generate data** that **implicitly adapts to** the p_{data} distribution

Assume to have:

- A **prior density** $p_\omega \in \Delta(\Omega)$
- A **generator** (a decoder) $g_\theta \in X^\Omega$

The **generator** g_θ **generates datapoints** in X from Ω



The **system** of a GAN is based on **2 main component**:

- **Generator** g_θ → **generates fake data**
- **Discriminator** t_φ → **detects** whether **data** is **real or generated** by the generator

We **aim** to **compute** $q_\theta(x)$

- It's the **global probability** that our **model generates** the **output** x → aim too **maximize it**

$$q_\theta(x) = \mathbb{E}_{\omega \sim p_\omega}[\delta(\omega) - x] \quad \delta \text{ is the Dirac delta function}$$

- We want it **to be as similar as** p_{data} as possible
- We **need to find the optimal parameters** θ
  such that the **Jensen-Shannon divergence** between q_θ and p_{data} is **minimized**

$$\theta^\star \in \arg\min_\theta d_{JS}(p_{data}, q_\theta)$$

Intractable objective

We have seen that $q_\theta(x) = \mathbb{E}_{\omega \sim p_\omega}[\delta(\omega) - x]$

However:

- Is an **infinite calculation** → it **can't** be **computed**
- It's **gradients can't** be **computed**

Finding a lower bound

$d_{JS}(p_{data}, q_\theta)$ can be **written** in an **equivalent form**:

- **Adding** a function $t(x)$ that behaves as **binary classifier** (defined as **discriminator**)
  - Trained to **predict** whether a data x is **from**
    - The **real distribution** p_{data} → returns True
    - The **generated one** q_θ → returns False

We can compute that:

$$d_{JS}(p_{data}, q_\theta) \geq \max_\varphi \left\{ \mathbb{E}_{x \sim p_{data}}[\log t_\varphi(x)] + \mathbb{E}_{x \sim q_\theta}[\log(1 - t_\varphi(x))] \right\}$$

However **the bound still depends on the explicit density** q_θ

We can **rewrite**:

$$\begin{aligned} d_{JS}(p_{data}, q_\theta) &\geq \max_\varphi \left\{ \mathbb{E}_{x \sim p_{data}}[\log t_\varphi(x)] + \mathbb{E}_{x \sim q_\theta}[\log(1 - t_\varphi(x))] \right\} \\ &= \max_\varphi \left\{ \mathbb{E}_{x \sim p_{data}}[\log t_\varphi(x)] + \mathbb{E}_{\omega \sim p_\omega}[\log(1 - t_\varphi(g_\theta(\omega)))] \right\} \end{aligned}$$

The **final objective** is:

$$\theta^\star \in \arg\min_\theta \max_\varphi \left\{ \mathbb{E}_{x \sim p_{\text{data}}}[\log t_\varphi(x)] + \mathbb{E}_{\omega \sim p_\omega}[\log(1 - t_\varphi(g_\theta(\omega)))] \right\}$$

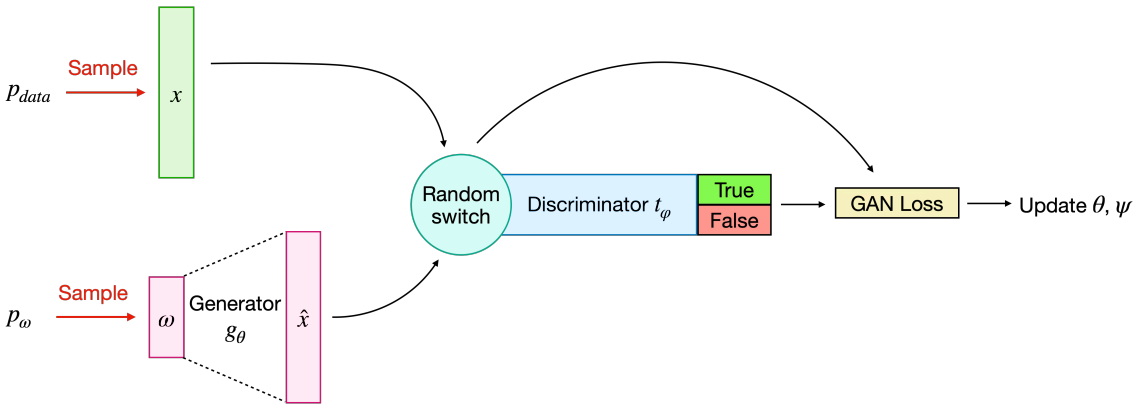
Observe that:

- The **discriminator** (parameters φ) aims to **maximize the function**
- The **generator** (parameters θ) aims to **minimize the function**

$x \sim p_{data}$ and $\omega \sim p_\omega$ are **obtained via sampled mini-batches**

Training phase

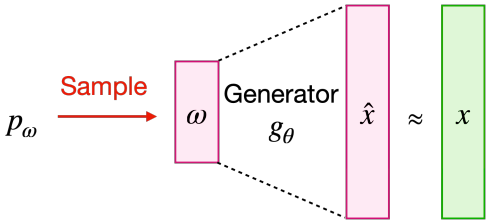
Based on the previous observations, we can define the training process of GAN



Schema of the training process

Inference phase

Based on the previous observations, we can define the inference process of GAN



Schema of the inference process

Advantages and issues

GANs have both advantages and issues

Advantages

- **No blurry** data
  - If the generator produces blurry data, they are **identified as fake by the discriminator**
  - → The **generator has to learn details and realistic textures**
- **Expressive latent space** → it **is linearly structured**
  - Allows for "**vectorial algebra**"
  - For example: **MAN WITH GLASSES - MAN + WOMAN = WOMAN WITH GLASSES**

Issues

- **Training instability**: the **loss** is based on a **Min-Max dynamic**
  - If the **generator improves**, the **loss** of the **determinator increases**
  - Il the **discriminator improves**, the **loss** of the **generator increases**
- **Mode collapse**
  - Assume a specific **sample** successfully **fools** the **discriminator**
    - **The generator** learns it fools the discriminator
    - It **will produce** always **data** in the **neighborhood of** the **fooling** data
  - ⚠️ The **model outputs** always the **exact data**, **losing** the **diversity** of the training set
- **Vanishing gradient**
  - In **early training**:
    - The **generator performs bad**
    - The **discriminator learns fast**
  - ⚠️ If the **discriminator learns too fast** his **accuracy** gets **too high**
    - The discriminator function gets plain
      - The **gradient** becomes smaller and smaller, closer to **zero**
      - The **generator** remains **stuck** as the discriminator gradient is extremely small

GANs evolution

Several versions of GAN exist to cope with their issues:

- Wasserstein GAN (= WGAN)
  - Doesn't rely on the probability
  - Measures the geometric distance between the distribution
- Hybrid architectures that combine VAEs with GANs (such as VAE-GAN)